ARCHITECTURE AGENTIQUE AUTONOME

SPÉCIFICATION TECHNIQUE & BLUEPRINT SYSTÉMIQUE MULTI-BOUCLES LOCAL

MASTER BLUEPRINT V2.3 (EXHAUSTIF & INTÉGRAL)

Résumé Exécutif : Ce document régit l'installation globale du système IA autonome local (`dsh-core`). Il orchestre les modèles spécialisés (Qwen Coder 14B/32B, DeepSeek 32B/R1, Mistral, Llama Vision, Flux/ComfyUI, Llama 3.2 3B) via le harnais `dsh`, gère la mémoire multi-niveaux et persistante (Mem0 + AnythingLLM + cold_storage), intègre les sous-agents d'exécution (dont le sous-agent de recherche web autonome `ResearchAgent`), connecte les sources de données (Google Drive Multi-Comptes, OneDrive, VS Code, n8n) et assure l'observabilité et le contrôle qualité en temps réel (Langfuse + React Flow + LLM-as-a-Judge).

1. CARTOGRAPHIE DES 3 BOUCLES AGENTIQUES & COCKPITS UI

1. Cockpit & UI

- **VS Code / CLI :** Dev direct & contrôle `dsh-core`.
- **AnythingLLM :** Chat, RAG local & mémoire froide.
- **n8n Workflows :** Automatismes, webhooks & triggers.

2. Boucle Verte

- **LLM-as-a-Judge :** Contrôle qualité & re-try loop.
- **Langfuse :** Traces & suivi des tokens/latence.
- **React Flow :** Dashboard visuel second écran.

3. Boucle Orange

- **dsh-core :** Orchestrateur & Router.
- **ResearchAgent :** Recherche web & scraping.
- **MCP Server :** Drive, OneDrive, Git.
- **ComfyUI :** Génération Flux/SD.

4. Boucle Jaune

- **Mem0 :** Faits clés & préférences.
- **Llama 3.2 (3B) :** Résumé & compression contextuelle.
- **AnythingLLM :** Vector DB sur `cold_storage/`.

2. SCHÉMA SYSTÈME GLOBAL & FLUX D'EXÉCUTION (DÉTAIL DE LA PHOTO 2)



Figure 2.1 — Modèle de flux d'exécution éphémère (RAM), boucle d'outils, persistance sémantique et contrôle LLM Ops.

Décomposition technique intégrale du schéma :

- **Enveloppe Harnais (Harness Scope) :** Le moteur `dsh-core` englobe la totalité de la session. Il maintient une étanchéité stricte entre l'espace d'exécution temporaire et le stockage froid d'archivage.
- **Boucle Agentique Éphémère (RAM) :** Tout est temporaire pendant la session. Le contexte est assemblé à partir de la demande utilisateur, de l'historique récent de chat et du prompt système, puis enrichi dynamiquement par :
  - `SKILL.md` (Mémoire procédurale / instructions de compétences et règles d'action).
  - DB Vectorielle (Mémoire sémantique / faits persistants récupérés via RAG top-k).
  - DB / SQL (Mémoire épisodique / événements récents et historique de conversations).
- **Boucle d'Outils (Tool Loop) :** L'Agent IA sollicite des outils dédiés (ex: `ResearchAgent`, workflows n8n, CRM, agenda, scripts de code). Les retours repassent par une couche de sécurité (Guardrails) avant la génération de la réponse finale à l'utilisateur.
- **Système de Compression de la Mémoire :** Dès le franchissement d'un seuil de N messages, l'agent ultra-léger `Llama 3.2 (3B)` supprime le superflu, extrait les faits essentiels et les enregistre dans la mémoire sémantique durable (DB Vectorielle / Mem0 / `cold_storage`).
- **Module LLM Ops & Porte de Qualité (LLM Ops Gate) :** Chaque exécution génère une trace complète ($1\text{ Run} = 1\text{ Trace}$) envoyée vers Langfuse. Le composant *LLM-as-a-Judge* évalue la conformité du résultat :
  - *Si non valide :* Diagnostic d'erreur (pourquoi/quand ?) et relance automatique avec escalade vers un modèle supérieur.
  - *Si valide :* Passage de la Gate, mise en production sécurisée et ajustement du prompt système / RAG $top\text{-}k$.

3. MATRICE EXHAUSTIVE DES MODÈLES IA DU SYSTÈME

Le système repose sur un pool dynamique de modèles spécialisés basculant selon la complexité :

Modèle & Taille	Hébergement	Spécialité & Usage Principal	Niveau de Cascade
Qwen 2.5 Coder 14B	Local (Ollama / vLLM)	Code rapide, scripts, formatage JSON, requêtes courantes.	NIVEAU 1 (Par défaut)
Mistral 7B/12B	Local (Ollama)	Rédaction de contenus, synthèses de texte, emails, tri.	NIVEAU 1 (Généraliste)
Qwen 2.5 Coder 32B	Local (Ollama / vLLM)	Architecture logicielle, refactoring lourd, workflows n8n complexes.	NIVEAU 2 (Escalade Code)
DeepSeek 32B / R1	Local / OpenRouter	Raisonnement complexe, planification de tâches multi-étapes.	NIVEAU 2 (Planner / Agent)
Llama 3.2 Vision	Local (Ollama)	Lecture de documents scannés, OCR, tickets de caisse, schémas.	NIVEAU 2 (Analyse Vision)
Flux / SD (ComfyUI)	Local (ComfyUI MCP)	Génération d'images pour catalogues, visuels web, rendus.	NIVEAU 2 (Génération Image)
Llama 3.2 (3B)	Local (Ollama)	Sous-agent ultra-rapide d'arrière-plan pour la compression de mémoire Mem0 et résumés.	INFRA (Boucle Jaune)
ResearchAgent	Local (dsh-core)	Sous-agent de collecte web autonome, scraping et structuration d'infos.	INFRA (Boucle Orange)
OpenRouter (Claude / GPT)	API Distante	Secours ultime en cas d'échec répété des modèles locaux.	NIVEAU 4 (Fallback Ultime)

4. LOGIQUE DE ROUTAGE EN CASCADE & CONTRÔLE UTILISATEUR

A. Routage Automatique par Escalade (Fallbacks) : Le harnais `dsh` applique un principe strict d'économie de ressources VRAM :

- **Démarrage ultra-rapide :** La tâche est d'abord confiée au modèle le plus léger (ex: Qwen 14B).
- **Détection d'échec (Boucle Verte) :** Le composant *LLM-as-a-Judge* vérifie le résultat (syntaxe, critères respectés, format JSON).
- **Escalade automatique :** En cas d'erreur ou de rejet, le harnais réinjecte le journal d'erreur et passe la tâche au modèle supérieur (32B ou DeepSeek R1).
- **Dernier recours :** Si le 32B local échoue, bascule transparente vers l'API OpenRouter (Claude / GPT-4o).

B. Sélection Manuelle (User Override) : L'utilisateur conserve la priorité absolue à tout moment :

- **Sélecteur UI :** Dans VS Code (Continue) ou AnythingLLM, un champ permet d'imposer un modèle spécifique pour une session.
- **Profils par Projet :** Possibilité de définir des règles strictes (ex: "Projet X = 100% Qwen 14B pour préserver la VRAM GPU").
- **Déclenchement direct d'outils :** Dire "Recherche sur le web..." active directement `ResearchAgent` sans passer par l'escalade textuelle.

5. CONNECTEURS CLOUD & INTÉGRATION DOCUMENTAIRE

Afin de permettre à l'agent de retrouver et d'analyser tous tes documents sans import manuel :

- **Google Drive MCP (Multi-comptes) :** Branchement simultané des comptes professionnels et personnels via l'API Google Workspace. L'agent peut rechercher des fichiers par nom, contenu, date ou dossier.

- **OneDrive MCP :** Connecteur Microsoft Graph permettant l'exploration des dossiers Cloud OneDrive, la lecture de tableurs Excel, et la synchronisation locale.
- **RAG Hybride (AnythingLLM) :** Les documents et archives générés dans `A:\dsh-core\memory\cold_storage\` sont indexés automatiquement dans la base vectorielle locale pour des recherches sémantiques instantanées.

6. OBSERVABILITÉ VISUELLE EN TEMPS RÉEL (ÉCRAN SECONDARY DASHBOARD)

Pour sortir de l'effet "boîte noire", l'architecture intègre un double système de suivi en direct :

- **A. Langfuse (Docker) — Traçabilité & Logs Fine :**
  - *Arbre d'exécution (Traces) :* Visualisation exacte de chaque appel de fonction et de chaque sous-agent.
  - *Statistiques en direct :* Nombre de tokens consommés, modèle sollicité (14B vs 32B), latence en millisecondes.
  - *Inspecteur MCP :* Visualisation des données brutes envoyées/reçues depuis Google Drive, n8n ou le système de fichiers.
- **B. Schéma Animé (React Flow / Langflow) :**
  - *Surbrillance Dynamique :* Chaque fois qu'une boucle s'active, la case correspondante sur le schéma s'allume (ex: Jaune quand Mem0 écrit, Orange quand `ResearchAgent` ou n8n s'exécutent).
  - *Impulsions Lumineuses :* Des flux animés le long des flèches montrent le déplacement des données en temps réel.
  - *Alerte d'Échec :* Clignotement rouge immédiat lors d'un rejet par la Boucle Verte suivi de l'animation d'escalade vers le modèle 32B.

7. SPÉCIFICATIONS TECHNIQUES DES COMPOSANTS CLÉS

- **DeepSeek Harnais (`dsh-core`) :** Moteur d'orchestration central situé dans `A:\dsh-core\`, pilotable via CLI, n8n ou AnythingLLM. Il découpe le prompt en plan d'action, choisit le modèle et émet des événements WebSocket en direct vers Langfuse.
- **Sous-Agent `ResearchAgent` (`agents/research_agent.py`) :** Script Python de collecte web autonome. Il effectue les requêtes réseau, extrait le texte brut, le fait structurer en JSON par `llama3.2:3b` et l'archive en mémoire froide.
- **Mem0 + Llama 3.2 3B (Moteur de Mémoire Persistante) :** Évite la saturation du contexte. Régulièrement, le sous-agent `Llama 3.2 3B` extrait les faits durables et les stocke dans la base sémantique.
- **Moteur de Workflows n8n & Serveurs MCP :** Le bras armé du système pour les actions dans le monde réel (scraping, envoi de mails, gestion de factures, automatismes).

8. FEUILLE DE ROUTE D'INSTALLATION INTÉGRALE

Étape	Composant & Action	Détails d'Exécution & Configuration	Statut
1.1	Configuration VS Code + MCP Host	Finalisation du serveur MCP local, accès fichiers et terminal.	TERMINÉ
1.2	Structure dsh-core & Mémoire Multi-Niveaux	Création de l'arborescence A:\dsh-core et gestionnaire de mémoire.	TERMINÉ
1.3	Intégration Llama 3.2 (3B)	Configuration d'Ollama local pour résumés et compression.	TERMINÉ
1.4	Brique ResearchAgent (agents/research_agent.py)	Sous-agent de recherche web et structuration automatique JSON.	EN COURS
2.1	Google Drive & OneDrive MCP	Authentification OAuth multi-comptes Google et Graph API.	EN COURS
2.2	Base RAG AnythingLLM & Cold Storage	Connexion d'AnythingLLM au sous-dossier memory/cold_storage/.	EN COURS
3.1	DeepSeek Harnais & Cascade	Déploiement de dsh, règles de routage (14B -> 32B -> OpenRouter).	À FAIRE
3.2	Bridge n8n & ComfyUI MCP	Exposition des Webhooks n8n et du serveur de génération Flux/SD.	À FAIRE
3.3	Dashboard Animé & Langfuse	Lancement de Langfuse en Docker et schéma React Flow sur Écran 2.	À FAIRE
4.1	Boucle Verte & Registre Skills	Mise en place du validateur LLM-as-a-Judge et auto-sauvegarde SKILL.md.	À FAIRE

9. EXEMPLES CONCRETS D'EXÉCUTION DU SYSTÈME

Scénario 1 : Recherche Web Autonome & Veille Technique

- 1. *Réception :* L'utilisateur lance `python dsh_harness.py --research "Nouveautés Ollama 2026"` ou déclenche un webhook n8n.
- 2. *Appel Outil (Boucle Orange) :* `ResearchAgent` effectue les requêtes web, récupère les pages et les transmet à `Llama 3.2 (3B)`.
- 3. *Traitement & Mémoire (Boucle Jaune) :* Llama 3.2 génère une synthèse JSON propre enregistrée dans `memory/cold_storage/research_20260919.json`.
- 4. *Consultation (Cockpit UI) :* AnythingLLM indexe le fichier. L'utilisateur peut discuter directement avec sa nouvelle base de connaissances.

Scénario 2 : Recherche Immobilière Multi-Critères

Injonction : "*Trouve-moi la liste des appartements T3/T4 à vendre sur [Ville] < 250k€ avec terrasse, mets-les dans un tableau propre et fais une synthèse.*"

- 1. *Réception Prompt :* Le harnais `dsh` attribue la tâche initiale à Qwen 14B (rapide).
- 2. *Appel Outil (Boucle Orange) :* Qwen 14B déclenche le workflow n8n de scraping des annonces et interroge Google Drive / OneDrive pour vérifier les fiches déjà sauvegardées.
- 3. *Traitement & Contrôle (Boucle Verte) :*
  - Si Qwen 14B génère le tableau parfait -> Validé par le Judge.
  - Si Qwen 14B rate un critère (ex: oubli de la terrasse) -> Le Judge rejette le résultat, le harnais bascule automatiquement vers Qwen 32B / DeepSeek R1 qui corrige et finalise la réponse.
- 4. *Mémoire (Boucle Jaune) :* Mem0 extrait tes critères (T3/T4, Ville, < 250k€) pour s'en souvenir lors de tes prochaines sessions.
- 5. *Observabilité (Écran 2) :* Le schéma React Flow s'allume en direct à chaque étape et Langfuse enregistre le coût exact de la requête.

10. PRINCIPES DE MAINTENANCE & ÉVOLUTIVITÉ

- **Remplacement d'un Modèle :** Pour intégrer un nouveau modèle (ex: Qwen 3 ou Llama 4), il suffit de modifier son nom dans le fichier de config du harnais `dsh`. Aucun workflow n8n ou serveur MCP n'a besoin d'être réécrit.
- **Ajout de Comptes Cloud :** Il suffit de rajouter un jeton d'accès OAuth dans le serveur MCP Google Drive / OneDrive pour étendre instantanément le périmètre de recherche de l'agent.
- **Persistance de la Mémoire :** Les répertoires de mémoire (`cold_storage`), AnythingLLM, les bases Mem0 et les fichiers `SKILL.md` sont stockés en local sur le disque et survivent à toutes les mises à jour logicielles.